CONVOLUTION KERNELS STUDY NOTES

ARI2129 – Principles of Computer Vision for AI

Dusan Dinic, Dimitrios Paschalidis, Nathan Tanti, Gregory Vella, James Vella-Gera
Group 9

Contents

1. The "Why" and "How"

To understand a convolution kernel, one must first understand the limitations of a computer's vision. A computer does not 'see' an image as a cohesive whole; it sees a massive grid of numbers representing light intensity. In a standard digital photo, there might be up to millions of these numbers. For a machine to make sense of this chaos, it needs a way to filter the signal from the noise. This is where the kernel comes into play.

The Flashlight Analogy: Finding the Signal

Imagine you are in a pitch-black cathedral, searching for a specific architectural detail - let's say, a Gargoyle's wing. You have a small flashlight that only illuminates a 1 square meter section of the wall. As you move this light across the cathedral, you are scanning for specific shapes. If you are looking for vertical lines (the Gargoyle's wing feathers), your brain is effectively 'filtering' out the horizontal lines of the stone blocks. The light is the kernel. The map of where you found the wings is the feature map. Because you are using the same flashlight (kernel) for the whole wall, if you learn how to identify a wing at the bottom of the wall, you can identify it at the top as well. This is 'weight sharing,' and it is the single most important concept in efficient image recognition [2].

Local vs. Global Context

Consider a chef tasting a massive pot of stew. To understand the flavor, the chef doesn't drink the whole pot at once. Instead, they take a small spoonful from the surface, then another from the bottom. Each spoonful is a 'local sample.' If the chef finds a piece of salt in one spoon, they know there is salt in that region. By taking many small samples across the pot, they build a 'map' of the stew's composition. In a CNN, the first layer takes small 'spoonfuls' (3x3 kernels) to find salt and pepper (edges). The next layer takes 'bowl-fuls' (combining multiple kernels) to find carrots and potatoes (shapes). Finally, the last layer understands the whole stew (the object). This hierarchy allows the model to build complex understanding from simple observations.

2. Technical Foundations: The Mathematical Engine

Convolution is the process of applying a filter to an input to get an output. In 2D space, this involves a sliding window operation where the kernel weights are multiplied by the underlying input pixels, and the results are summed into a single value.

The Universal Dimension Equation

Predicting the output volume is essential for avoiding memory errors and ensuring layers connect correctly. For an input of dimension I, kernel K, padding P, and stride S, the output O is:

$$O = [(I - K + 2P) / S] + 1$$

Let us examine the sensitivity of each parameter:

- **Kernel Size (K):** Larger kernels like 11x11 capture more context but are computationally 'heavy.' Smaller kernels (3x3) allow for deeper networks, which can learn more complex patterns [5, 6].
- **Stride (S):** The 'compression' factor. S=1 keeps the image size stable. S=2 halves the size, effectively acting as a 'learnable pooling' layer [5, 6].
- **Padding (P):** The 'boundary keeper.' Zero-padding ensures that the information at the corners of the image isn't lost. Without it, data 'shrinks' with every layer.
- **Dilation (d)**: Dilation introduces gaps between kernel weights, allowing a small kernel to see a massive area without increasing the number of calculations

3. The Specialist Families: Sobel, Gaussian, and Laplacian

Classical kernels are the 'ancestors' of modern deep learning filters. While AlexNet and MobileNet learn their kernels, they often converge on weights that look remarkably like these classical filters.

The Sobel Family: Edge Detection

The Sobel operator is used to detect horizontal and vertical gradients. By finding areas where the pixel brightness changes rapidly, it identifies the outlines of shapes.

The standard 3×3 Sobel kernels are:

Sobel X (detects vertical edges):

$$\begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

Sobel Y (detects horizontal edges):

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

The combined edge magnitude is calculated as:

$$\textbf{magnitude} = \sqrt{(Gx^2 + Gy^2)}$$

where Gx and Gy are the outputs of the X and Y kernels respectively.

In the 1998 LeNet model, these edge detectors were the first step in recognizing the strokes of a handwritten '7' or '3' [3].

The Gaussian Family: Smoothing

Gaussian kernels act as low-pass filters. They average the surrounding pixels using a bell-curve weight, effectively blurring the image.

A standard 3×3 Gaussian kernel (σ≈0.85) is:

$$\begin{bmatrix} 1, 2, 1 \\ 2, 4, 2 \\ 1, 2, 1 \end{bmatrix} \div 16$$

The centre weight (4) has the highest influence, meaning pixels closer to the centre contribute more to the output than those at the corners (1).

This is used to remove 'noise', the random graininess in photos, so that the model doesn't get distracted by irrelevant details.

The Laplacian Family: Detail and Contrast

The Laplacian kernel is a second-order derivative. It identifies the 'edges of edges.'

The standard 3×3 Laplacian kernel is:

$$\begin{bmatrix} 0, 1, 0 \\ 1, -4, 1 \\ 0, 1, 0 \end{bmatrix}$$

The negative centre (−4) fires strongly wherever surrounding pixels differ significantly from the centre pixel, making it highly sensitive to edges in all directions simultaneously.

It is extremely sensitive to fine details and is used in tasks like image sharpening or medical scan analysis where every micro-texture matters.

> **Interactive exploration:** Open the Convolution Kernel Simulator and load the Gradient sample image. Switch between Sobel X, Sobel Y, and Laplacian. Notice how Sobel Y produces a near-black output — because there is no vertical change in a horizontal gradient. The Laplacian similarly produces near-black, as the gradient changes at a constant rate and the second derivative is close to zero.

4. Worked Numerical Example: A Full Calculation

Let us compute a full 2×2 output from a 4×4 input using a 3×3 Identity kernel with Stride=1 and Padding=0.

Input (I):

$$\begin{bmatrix} 1, 1, 1, 0 \\ 1, 1, 1, 0 \\ 1, 1, 1, 0 \\ 0, 0, 0, 0 \end{bmatrix}$$

Kernel (K) — Identity Filter:

$$\begin{bmatrix} 0, 0, 0 \\ 0, 1, 0 \\ 0, 0, 0 \end{bmatrix}$$

Output size verification: $O = (4 - 3 + 0)/1 + 1 = \mathbf{2}$, so we expect a 2×2 output with 4 positions total.

Step 1: Position (1,1) — Top-Left

The kernel is placed over the top-left 3×3 region:

$$\begin{bmatrix} 1, 1, 1 \\ 1, 1, 1 \\ 1, 1, 1 \end{bmatrix}$$

Calculation: $(0\times1)+(0\times1)+(0\times1)+(0\times1)+(1\times1)+(0\times1)+(0\times1)+(0\times1)+(0\times1) = \mathbf{1}$

Step 2: Position (1,2) — Top-Right

Slide the kernel one step to the right:

$$\begin{bmatrix} 1, 1, 0 \\ 1, 1, 0 \\ 1, 1, 0 \end{bmatrix}$$

Calculation: $(0\times1)+(0\times1)+(0\times0)+(0\times1)+(1\times1)+(0\times0)+(0\times1)+(0\times1)+(0\times0) = \mathbf{1}$

Step 3: Position (2,1) — Bottom-Left

Slide the kernel one step down from Position (1,1):

[1, 1, 1]
[1, 1, 1]
[0, 0, 0]

Calculation: (0×1)+(0×1)+(0×1)+(0×1)+(1×1)+(0×1)+(0×0)+(0×0)+(0×0) = **1**

Step 4: Position (2,2) — Bottom-Right

Slide the kernel one step right from Position (2,1):

[1, 1, 0]
[1, 1, 0]
[0, 0, 0]

Calculation: (0×1)+(0×1)+(0×0)+(0×1)+(1×1)+(0×0)+(0×0)+(0×0)+(0×0) = **1**

Final Output Map:

[1, 1]
[1, 1]

All four positions produce 1 because the identity kernel simply picks the centre pixel, and all centre pixels in this input are 1. The result confirms $O = (4 - 3 + 0)/1 + 1 = 2$.

> **Try it yourself:** Open the Convolution Kernel Simulator (simulator/app.py in the repository), select the Checkerboard sample image, choose Gaussian with Kernel Size=3, Padding=0, Stride=1. The output dimensions displayed should read 254×254 — confirming the formula $O = (256 - 3 + 0)/1 + 1 = 254$.

5. The Evolution of the Kernel

The history of the kernel is a journey from rigid mathematics to flexible intelligence. It began with Fukushima's Neocognitron, which tried to copy the human brain's visual pathways [1]. It matured with Yann LeCun's LeNet, which proved that we could train these networks to read bank checks automatically [3]. The modern era arrived with AlexNet, which showed that if we make kernels 'deep' enough and run them on fast GPUs, they can recognize almost anything [5].

Today, we focus on efficiency. MobileNets [7] and Xception [8] use 'Depthwise Separable Convolutions.' Imagine instead of using a big 3x3 paintbrush that uses 3 colors at once, you use a small 3x3 brush for each color independently, then a tiny 1x1 brush to mix them. This 'Separable' method is much faster and energy efficient.

6. Quick Reference (Cheat Sheet)

Key Equations at a Glance

Objective	**Formula / Rule**
Output Dimension	(I - K + 2P)/S + 1
Effective Receptive Field	R_{layer-i} = R_{i-1} + (K_i - 1) * Stride_cumulative
Depthwise Separable Speedup	~1/N + 1/K²
Same Padding (S=1)	P = (K-1)/2
Parameters (Conv Layer)	K * K * Channels_In * Channels_Out

Parameter Ranges

Parameter	**Typical Range**	**Notes**
Kernel Size (K)	3, 5, 7 (always odd)	Larger = more context, more cost
Sigma (Gaussian)	0.5 – 3.0	Higher = Stronger blur
Stride (S)	1 – 4	S = 1 is standard, S = 2 halves output
Padding (P)	0 or (k - 1) / 2	0 = valid, (k-1)/2 = same
Dilation (d)	1 – 8	d = 1 is standard convolution

Glossary and Misconceptions

Stride: The step size of the kernel. High stride = high compression.

Feature Map: The 'activation' output. Think of it as a heatmap of patterns.

Dilation: Gaps in the kernel. High dilation = wide view with few parameters.

Pointwise Conv: A 1x1 kernel. Used to change the number of channels (depth).

Common Misconceptions:

"Larger kernels always capture more detail." False. Larger kernels increase the receptive field but reduce spatial resolution, increase computation cost, and can cause more information loss at boundaries. Stacking smaller kernels (e.g. two 3×3 layers) achieves the same receptive field as one 5×5 with fewer parameters.

"Padding adds real image information." False. Zero-padding adds artificial zeros. It preserves spatial size and ensures corner pixels are visited as often as centre pixels, but it introduces boundary artifacts, kernels near the edge compute against values that do not exist in the original image.

"Stride and pooling do the same thing." False. Pooling uses a fixed rule (max or average) to downsample. Strided convolution learns what to compress through backpropagation, it has trainable weights. Strided convolution is therefore more flexible but also more computationally expensive to train.

7. Key Papers: Annotated Bibliography

[1] Fukushima, K. (1980). Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position. Biological Cybernetics.

Inspired by Hubel and Wiesel's Nobel-winning work on the cat's visual cortex, Fukushima proposed a model of 'S-cells' (detecting local features) and 'C-cells' (handling positional shifts). Unlike modern networks, it used unsupervised learning. It proved that a hierarchy of kernels could achieve 'shift invariance', recognising a character regardless of its exact position in the input.

Direct URL: https://www.rctn.org/bruno/public/papers/Fukushima1980.pdf

Direct Relevance: Established the kernel hierarchy architecture used by every modern AI.

---

[2] Waibel, A., Hanazawa, T., Hinton, G., Shikano, K., & Lang, K. J. (1989). Phoneme Recognition Using Time-Delay Neural Networks. IEEE Transactions.

This paper applied kernels to 1D audio, introducing Time-Delay Neural Networks (TDNN). A sliding kernel detected phonemes, the building blocks of speech, solving the problem of temporal shifts (a word starting at 0.1s vs 0.5s). The TDNN achieved 98.5% accuracy, beating the industry-standard Hidden Markov Models of the time.

Direct URL: https://www.cs.toronto.edu/~fritz/absps/waibelTDNN.pdf

Direct Relevance: Proved that kernels are universal feature extractors for any sequential or spatial data.

---

[3] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE.

LeNet-5 was the first modern CNN, legendary for two reasons: it integrated backpropagation with convolutional layers, and it delivered a real-world application, reading bank checks and zip codes for the US Postal Service. The Graph Transformer Network concept allowed the entire pipeline to be trained end-to-end to minimise error rate.

Direct URL: http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf

Direct Relevance: The blueprint for all commercial OCR and early image recognition systems.

---

[4] Haussler, D. (1999). Convolution Kernels on Discrete Structures. Technical Report UCSC-CRL-99-10.

Haussler extended kernel theory to discrete structures like DNA strings, linguistic trees, and chemical graphs. Using 'R-convolutions,' he proved that any object decomposable into simpler parts can have a valid kernel built for it. This work is foundational for bioinformatics and Natural Language Processing.

Direct URL: https://bpb-us-e1.wpmucdn.com/sites.ucsc.edu/dist/4/821/files/2018/12/Convolution-Kernels.pdf

Direct Relevance: Expanded kernel theory to non-visual, highly structured discrete data sets.

---

[5] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. NIPS.

AlexNet crushed the ImageNet competition using large kernels (11×11), GPU acceleration, and the ReLU activation function. It established that 'Deeper is Better,' using 8 layers and 60 million parameters. Dropout was introduced to prevent overfitting. Before this paper, CNNs were considered outdated, AlexNet single-handedly reversed that perception.

Direct URL: https://proceedings.neurips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

Direct Relevance: Kicked off the deep learning era and made GPUs the standard for AI training.

---

[6] Yu, F., & Koltun, V. (2016). Multi-Scale Context Aggregation by Dilated Convolutions. ICLR.

Semantic segmentation requires both global context and fine detail simultaneously. Yu and Koltun solved this with Dilated Convolutions, spreading kernel weights apart to see a

massive area while keeping computation small. Their model outperformed every other entry on the Cityscapes self-driving dataset.

Direct URL: https://arxiv.org/abs/1511.07122

Direct Relevance: The fundamental technique for high-resolution scene understanding and medical imaging.

---

[7] Howard, A. G., et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv.

Standard convolutions are too slow for mobile devices. MobileNets introduced Depthwise Separable Convolutions, splitting spatial filtering from channel-mixing, reducing computational cost by 90% with only a 1% accuracy drop. Width Multipliers were also introduced, allowing kernels to be scaled to fit specific hardware constraints.

Direct URL: https://arxiv.org/abs/1704.04861

Direct Relevance: Enabled high-quality AI to run on smartphones and embedded devices.

---

[8] Chollet, F. (2017). Xception: Deep Learning with Depthwise Separable Convolutions. CVPR.

Chollet hypothesised that the most efficient approach is to fully decouple spatial correlations from cross-channel correlations. Xception applies depthwise separable convolutions throughout the entire network. This extreme approach outperformed Google's own Inception-v3 on large-scale datasets while maintaining the same model size.

Direct URL: https://arxiv.org/abs/1610.02357

Direct Relevance: Refined the theory of efficient kernels and set a new benchmark for image classification.

Conclusion: The Future of the Kernel

As we look forward, the convolution kernel remains the vital part of spatial data processing. However, the nature of the kernel is changing. We are seeing the rise of 'Dynamic Kernels' that change their own weights based on the input image, effectively 'focusing' more on interesting areas. We are also seeing the integration of kernels with 'Attention Mechanisms' (Transformers), where the kernel identifies the feature and the attention layer decides how important that feature is globally. Whether it is in the pocket of a smartphone user, a self-driving car, or the workstation of a doctor, the convolution kernel continues to bridge the gap between raw data and meaningful insights.

> **Test your understanding:** Take the Adversarial Quiz to check your comprehension of the concepts covered in these notes. The Google Form link is available in quiz_link.txt in the repository.